

COBERTURA DE PRUEBAS

Este documento presenta un análisis detallado de la cobertura de pruebas unitarias, generado mediante la herramienta JaCoCo.

Contenido:

- Microservicio Clientes [customer-service]
- Microservicio Cuentas [account-service]

Microservicio de Clientes [customer-service]

Cobertura: 86% 

 customer-service

customer-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.nttdata.customerservice.service.strategy		70 %		42 %
com.nttdata.customerservice.exception		80 %		n/a
com.nttdata.customerservice		37 %		n/a
com.nttdata.customerservice.mapper		94 %		n/a
com.nttdata.customerservice.service.impl		98 %		100 %
com.nttdata.customerservice.controller		100 %		n/a
com.nttdata.customerservice.config		100 %		n/a
com.nttdata.customerservice.dto		100 %		n/a
com.nttdata.customerservice.model		100 %		n/a
Total	65 of 478	86 %	8 of 18	55 %

Run  customer-service [clean,test] x

  

 customer-service [clean,test]: 47 sec, 429 ms

 com.nttdata:customer-serv 43 sec, 696 ms

[INFO]

[INFO] Results:

[INFO]

[INFO] Tests run: 36, Failures: 0, Errors: 0, Skipped: 0

[INFO]

[INFO]

[INFO] --- jacoco:0.8.8:report (report) @ customer-service -

[INFO] Loading execution data file C:\Users\stephanie.azorsa

[INFO] Analyzed bundle 'customer-service' with 16 classes

[INFO] -----

[INFO] BUILD SUCCESS

[INFO] -----

[INFO] Total time: 44.412 s

[INFO] Finished at: 2025-09-15T11:29:41-05:00

[INFO] -----

Casos de prueba (CP) considerados:

CustomerServiceTest		
CP	Método / Endpoint / Manejador	Descripción
CP-CS01	getCustomers()	Retorna lista de clientes cuando existen
CP-CS02	getCustomers()	Retorna lista vacía cuando no hay clientes
CP-CS03	getCustomerById()	Retorna cliente cuando existe
CP-CS04	getCustomerById()	Lanza excepción cuando cliente no existe
CP-CS05	createCustomer()	Debe crear cliente cuando las validaciones pasan
CP-CS06	createCustomer()	Debe lanzar excepción cuando las validaciones fallan
CP-CS07	updateCustomer()	Debe actualizar cliente cuando las validaciones pasan
CP-CS08	updateCustomer()	Debe lanzar excepción cuando cliente no existe al actualizar
CP-CS09	updateCustomer()	Debe lanzar excepción cuando validaciones fallan
CP-CS10	deleteCustomer()	Elimina cliente cuando no tiene cuentas activas
CP-CS11	deleteCustomer()	Lanza excepción cuando cliente tiene cuentas activas
CustomerControllerTest		
CP-CS12	createCustomer POST /customers	Retorna 201 created con cliente creado
CP-CS13	getCustomers GET /customers	Retorna 200 OK con lista de clientes
CP-CS14	getCustomerById GET /customers/{id}	Retorna 200 OK con cliente específico
CP-CS15	updateCustomer PUT /customers/{id}	Retorna 200 OK con cliente actualizado
CP-CS16	deleteCustomer DELETE /customers/{id}	Retorna 204 No Content
CP-CS17	getCustomers GET /customers	Retorna 200 OK con lista vacía
CP-CS18	getCustomerById GET /customers/{id}	Retorna 404 Not Found lanza una excepción si el cliente no existe
CP-CS19	createCustomer POST /customers	Retorna 400 lanza una excepción si DNI existe
CP-CS20	updateCustomer PUT /customers/{id}	Retorna 404 Not Found lanza una excepción si el cliente no existe
GlobalExceptionHandlerTest		
CP-CS21	MethodArgumentNotValidException	Retorna 400 con errores de validación
CP-CS22	DniAlreadyExistsException	Retorna 400 con mensaje apropiado
CP-CS23	CustomerNotFoundException	Retorna 404 con mensaje apropiado
AccountValidationServiceTest		
CP-CS24	validateCustomerHasNoAccounts()	Debe lanzar excepción si tiene cuenta(s) asociadas
CP-CS25	validateCustomerHasNoAccounts()	Debe lanzar excepción si existe errores del servidor
CP-CS26	validateCustomerHasNoAccounts()	Debe lanzar excepción si existe errores de conexión
CP-CS27	validateCustomerHasNoAccounts()	Debe lanzar excepción si el Response es null
CP-CS28	validateCustomerHasNoAccounts()	Debe lanzar excepción si el Response contenga el array vacío
ValidationContextTest		
CP-CS29	executeValidations()	Debe llamar a todas las estrategias en orden

CP-CS30	executeValidations()	Debe para en la primera excepción
CP-CS31	executeValidations()	Debe manejar las estrategias de lista vacía
CP-CS32	executeValidations()	Debe manejar el customerId nulo
DniValidationStrategyTest		
CP-CS33	validate()	Debe lanzar excepción cuando existe DNI para nuevo cliente
CP-CS34	validate()	Debe lanzar excepción si DNI existe para otro cliente
CP-CS35	validate()	Debe lanzar excepción si el DNI no existe

Microservicio de Cuentas [account-service]

Cobertura: 84% 

 account-service

account-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.nttdata.accountservice.util		84 %		100 %
com.nttdata.accountservice.service.impl		97 %		86 %
com.nttdata.accountservice.factory		63 %		40 %
com.nttdata.accountservice.exception		75 %		n/a
com.nttdata.accountservice.mapper		52 %		n/a
com.nttdata.accountservice		37 %		n/a
com.nttdata.accountservice.controller		100 %		n/a
com.nttdata.accountservice.model		100 %		n/a
com.nttdata.accountservice.dto		100 %		n/a
com.nttdata.accountservice.config		100 %		n/a
Total	83 of 551	84 %	5 of 22	77 %

Run m account-service [clean,test] x

  

✓ account-service [clean,test] 1 min, 28 sec, 58 ms

[INFO]

[INFO] Results:

[INFO]

[INFO] Tests run: 34, Failures: 0, Errors: 0, Skipped: 0

[INFO]

[INFO]

[INFO] --- jacoco:0.8.8:report (report) @ account-service ---

[INFO] Loading execution data file C:\Users\angie\Desktop\Nueva carpeta\services\account-service\target\jacoco.exec

[INFO] Analyzed bundle 'account-service' with 16 classes

[INFO] -----

[INFO] BUILD SUCCESS

[INFO] -----

[INFO] Total time: 01:23 min

[INFO] Finished at: 2025-09-15T17:30:32-05:00

[INFO] -----

Casos de prueba (CP) considerados:

AccountServiceTest		
CP	Método / Endpoint / Manejador	Descripción
CP-AS01	getAccounts()	Debe retornar lista de cuentas cuando existen
CP-AS02	getAccounts()	Debe retornar lista vacía cuando no hay cuentas
CP-AS03	getAccountById()	Debe retornar cuenta cuando existe
CP-AS04	getAccountById()	Debe lanzar excepción cuando no existe
CP-AS05	getAccountByCustomerId()	Debe retornar cuentas del cliente
CP-AS06	getAccountByCustomerId()	Debe retornar lista vacía si no tiene cuentas
CP-AS07	createAccount()	Debe crear cuenta cuando cliente existe
CP-AS08	createAccount()	Debe lanzar excepción cuando cliente no existe
CP-AS09	createAccount()	Debe validar saldo inicial mayor a 0
CP-AS10	createAccount()	Debe rechazar saldo inicial negativo
CP-AS11	deleteAccount()	Debe eliminar cuenta existente
CP-AS12	deleteAccount()	Debe lanzar excepción si cuenta no existe
TransactionServiceTest		
CP-AS13	deposit()	Debe aumentar saldo correctamente
CP-AS14	deposit()	Debe lanzar excepción si cuenta no existe
CP-AS15	deposit()	Debe rechazar depósito negativo
CP-AS16	deposit()	Debe rechazar depósito cero
CP-AS17	withdraw() [cuenta de ahorro]	Debe permitir retiro con saldo suficiente
CP-AS18	withdraw() [cuenta de ahorro]	Debe rechazar retiro con saldo insuficiente
CP-AS19	withdraw() [cuenta de ahorro]	Debe rechazar retiro que deje saldo negativo
CP-AS20	withdraw() [cuenta de corriente]	Debe permitir sobregiro dentro del límite (-500)
CP-AS21	withdraw() [cuenta de corriente]	Debe rechazar sobregiro fuera del límite (-500)
CP-AS22	withdraw() [cuenta de corriente]	Debe tener éxito en caso de límite exacto
AccountControllerTest		
CP-AS23	GET /accounts	Retorna 200 OK con lista de cuentas
CP-AS24	GET /accounts/{id}	Retorna 200 OK con cuenta
CP-AS25	GET /accounts/customer/{customerId}	Retorna 200 OK con cuentas del cliente
CP-AS26	POST /accounts	Retorna 201 Created con cuenta creada
CP-AS27	PUT /accounts/{accountId}/deposit	Retorna 200 OK con saldo actualizado
CP-AS28	PUT /accounts/{accountId}/withdraw	Retorna 200 OK con saldo actualizado
CP-AS29	DELETE /accounts/{id}	Debe retornar 204 No Content
GlobalExceptionHandlerTest		
CP-AS30	MethodArgumentNotValidException	Retorna 400 con errores de validación
CP-AS31	AccountNotFoundException	Retorna 400 con mensaje apropiado
CP-AS32	CustomerNotFoundException	Retorna 400 con mensaje apropiado
CP-AS33	InsufficientBalanceException	Retorna 400 con mensaje apropiado